

Weave *(HiveInfer-evo)*

Runtime-Adaptive Distributed LLM Inference Across Heterogeneous Apple Silicon Macs

Revised Master's Project Proposal — August 2026

1. Project Summary

Large language models are becoming powerful enough to run locally, but many useful models still require more memory than a single consumer computer can provide. Our original HiveInfer proposal tried to solve this by connecting different types of devices—GPUs, laptops, CPUs, and Jetson boards—and splitting a large model across them.

The original idea was:

Instead of shrinking a large model until it fits on one device, combine the memory and compute of several devices and run the model across them.

The original proposal therefore included peer-to-peer networking with Hivemind, multiple inference backends, Jetson support, automatic routing, quantization research, encrypted activation transfer, and fault tolerance.

Since that proposal was written, however, distributed local inference has developed very quickly. Apple now provides official distributed support in MLX, Exo has become a much more capable distributed inference system, and projects such as dnet, OptiQ, and prima.cpp already address several parts of heterogeneous model placement.

Therefore, simply building another system that **splits an LLM across several computers is no longer a sufficient research contribution.**

We propose narrowing HiveInfer to one platform—**Apple Silicon Macs**—and changing the research question.

The revised project asks:

When several different Macs are available, how should an LLM be divided between them based not only on their hardware, but also on what is

happening right now: available memory, context length, number of requests, measured processing speed, and network performance?

Weave will build a **runtime-adaptive scheduler** on top of Apple's existing MLX distributed infrastructure. Instead of rebuilding networking and inference from scratch, the project will focus on deciding **which Macs should participate and how many model layers each Mac should execute for the current workload**.

2. What Has Changed Since the Original Proposal?

This is the main reason the project needs to be redesigned.

2.1 Apple now provides an official distributed ML stack

The biggest change is that Apple itself now supports distributed machine learning across Macs.

MLX supports distributed execution across multiple physical machines. Its communication backends include TCP-based RING communication and JACCL, Apple's low-latency communication system using RDMA over Thunderbolt 5. Starting with macOS 26.2, Macs with appropriate Thunderbolt 5 hardware can communicate directly using RDMA. Apple also provides tools such as `mlx.launch` and `mlx.distributed_config` to configure and launch distributed workloads.

Apple now demonstrates large models running across multiple Macs using both pipeline and tensor parallelism. In its WWDC 2026 demonstration, Apple ran distributed inference across four M3 Ultra systems and demonstrated that models too large for one machine can be distributed across the cluster.

This means several things that were difficult engineering problems in our original proposal are now already available:

- communication between Macs,
- distributed model execution,
- process launching,
- Thunderbolt networking,
- collective communication,
- pipeline/tensor parallelism,
- and MLX-optimized Apple Silicon execution.

Effect on our project

We should **use this infrastructure rather than recreate it**.

Our contribution should sit *above* MLX and answer a problem MLX does not solve by itself: **how should a heterogeneous cluster be configured for a particular workload?**

3. Existing Systems Have Also Advanced

Apple is not the only important development.

Exo

Exo can connect devices automatically, uses MLX on Apple Silicon, supports RDMA over Thunderbolt, and automatically chooses distributed configurations using device resources and network topology. Its current system considers information such as available resources, network latency, and bandwidth.

Recent Exo development is going even further. Current work includes bandwidth-aware pipeline placement, reserving memory headroom before prefill, accelerator-aware heterogeneous placement, measuring link latency, and manually controlling pipeline layer allocation.

Therefore, we cannot claim that **hardware-aware or bandwidth-aware splitting itself is new**.

dnet

dnet is specifically designed for distributed inference across Apple Silicon clusters. It automatically discovers Macs, profiles devices and models, and has a heterogeneity-aware solver that considers compute performance, memory, network speed, KV-cache size, and even disk performance.

Therefore, we also cannot claim that **memory-aware heterogeneous model placement itself is new**.

prima.cpp

prima.cpp addresses heterogeneous consumer hardware and mathematically optimizes layer assignment based on computation, communication, memory, disk behavior, and operating-system differences.

Again, simply saying:

"We intelligently give faster computers more layers."

would not be novel enough.

OptiQ

OptiQ now provides distributed inference across multiple Apple Silicon Macs and can split large MLX models across machines with unequal amounts of memory.

Broader research

Workload-aware scheduling is also appearing in data-center research. For example, OServe dynamically changes LLM deployments according to changing workloads, while recent DOPS research studies closed-loop scheduling based on workload shape and runtime device contention.

This is important because we should **not claim that workload-aware scheduling as an idea has never been studied.**

Instead, our research gap must be much more precise.

4. How This Changes Hivelnfer

The original project tried to solve too many problems simultaneously.

Its architecture included Hivemind peer discovery, distributed routing, PyTorch/CUDA execution, llama.cpp for ARM/Jetson devices, CPU fallback, custom quantization, encrypted activations, redundant routing, KV-cache management, fault tolerance, and packaging.

For three students and approximately six to seven months, this is also unnecessarily broad now that mature infrastructure exists.

We therefore propose removing the following parts.

Original Hivelnfer Component	Revised Decision	Reason
NVIDIA/CUDA GPUs	Remove	We will focus only on Apple Silicon.

Jetson devices	Remove	Different backend and hardware ecosystem increases scope substantially.
CPU fallback	Remove as a research feature	Not central to the research question.
Hivemind DHT	Remove	MLX/Apple already provides distributed launching and communication.
Custom peer-to-peer networking	Remove	Existing infrastructure is sufficient.
llama.cpp backend	Remove	MLX will be the single execution backend.
TurboQuant research	Remove from primary scope	Quantization is now a separate and crowded research area.
Encrypted activation routing	Remove	Transport encryption does not protect activations from the machine that must process them; solving that properly would require an entirely different privacy project.
Automatic discovery	Not a contribution	Already available in systems such as Exo and dnet.
Basic heterogeneous splitting	Not a contribution	Already addressed by Exo, dnet, and prima.cpp.
Packaging / one-command installation	Optional final polish	Useful for demonstration, but not master's-level novelty.
Fault tolerance	Stretch goal only	Valuable, but secondary to the scheduler research.

This narrowing is intentional.

Instead of building ten different features at moderate depth, we want to solve **one difficult systems problem properly and evaluate it thoroughly**.

5. The Actual Research Gap

Suppose we have three Macs:

- Mac A — powerful GPU, 128 GB unified memory
- Mac B — medium GPU, 64 GB unified memory
- Mac C — slower GPU, 36 GB unified memory

A large model must be divided among them.

Existing systems can already look at relatively stable information such as:

- how much RAM each machine has,
- how powerful each machine is,
- network speed,
- model size,
- and sometimes KV-cache requirements.

That solves an important part of the problem.

However, **the best division is not determined only by the hardware.**

The best division can change depending on the request being processed and the current state of the machines.

For example:

Case 1 — Context changes

A user initially sends a 2,000-token prompt.

Later, the conversation contains 32,000 tokens.

The KV cache is now much larger. A Mac that had enough free memory earlier may become dangerously close to its memory limit.

The original model split may no longer be appropriate.

Case 2 — Concurrency changes

At first there is one request.

Later, four requests arrive simultaneously.

Memory requirements, batching behavior, and throughput requirements change.

The best configuration for one interactive user may not be the best configuration for several concurrent users.

Case 3 — Runtime performance changes

A Mac that is normally fast may currently:

- have another application consuming memory,
- be under heavy CPU/GPU load,
- experience thermal throttling,
- or simply execute its assigned pipeline stage slower than predicted.

A scheduler based only on the Mac model or total RAM will not capture all of this.

Case 4 — Using more Macs can actually hurt performance

If a model already fits comfortably on one powerful Mac, distributing it across three machines introduces communication overhead.

Therefore, the correct decision may sometimes be:

Do not distribute this request at all.

This decision is just as important as determining how to distribute one that needs multiple Macs.

6. Our Proposed Gap

Based on the systems and public documentation reviewed as of August 2026, we found strong solutions for:

- distributed execution,
- topology-aware placement,
- hardware profiling,
- memory-aware placement,
- network-aware placement,
- and heterogeneous layer assignment.

What we do **not** find clearly implemented as a complete Apple-Silicon system is a closed-loop scheduler that jointly uses:

1. the **current request characteristics**,
2. the **current state of every Mac**,

3. the **measured performance of the running pipeline**,
4. the **expected KV-cache/memory requirements**,
5. the **current concurrency level**,
6. the **cost of changing the model partition**,

and then decides:

Which Macs should run this workload, and how should the model be divided among them now?

This distinction is important.

We are **not claiming that dynamic scheduling itself has never been researched**. Related ideas exist in data-center and heterogeneous-accelerator research.

Our proposed contribution is narrower:

Design, implement, and experimentally evaluate runtime- and workload-aware adaptive model partitioning specifically for small heterogeneous Apple Silicon clusters using the modern MLX/JACCL stack.

The project will test whether using live workload and runtime information provides meaningful improvements over simpler placement methods.

7. Proposed Solution: Weave

Weave will sit **on top of MLX Distributed** rather than replacing it.

Conceptually:

Apple Silicon Macs

↓

MLX + MLX-LM

↓

MLX Distributed / JACCL / RING

↓

Weave Runtime Profiler

↓

Weave Adaptive Scheduler

↓

Chosen pipeline partition

The model itself will continue to run using MLX.

Our system's job is deciding **how to configure that execution**.

8. How Weave Will Work

Step 1 — Profile the Macs

When the cluster starts, Weave measures basic characteristics of each machine:

- available unified memory,
- model-layer execution speed,
- network bandwidth,
- network latency,
- and basic hardware information.

For example:

Device	Available Memory	Measured Layer Speed	Connection
Mac A	112 GB	Fast	TB5
Mac B	53 GB	Medium	TB5
Mac C	27 GB	Slower	Ethernet/TB

These measurements provide the initial hardware profile.

Step 2 — Understand the incoming workload

Before deciding how to run a request, Weave will examine information such as:

- current context length,
- requested maximum output length,
- number of concurrent requests,
- model being used,
- expected KV-cache requirement,
- and current memory headroom.

For example:

2K context + one user

is a very different workload from:

32K context + four concurrent users.

Step 3 — Observe actual runtime performance

Weave will also record what really happens during inference:

- how long each pipeline stage takes,
- which stage is slowing down the pipeline,
- how much memory each node is actually using,
- and whether network communication is becoming significant.

This is important because theoretical hardware specifications do not always represent actual performance.

Step 4 — Choose the number of Macs

The scheduler first asks:

Should this workload run on one, two, or three Macs?

For a smaller model, a single powerful Mac may be faster.

For a model that does not fit on one device, multiple machines become necessary.

For several simultaneous users, distributed execution may offer better overall throughput.

The system will therefore avoid assuming that **more devices automatically means better performance**.

Step 5 — Choose the layer distribution

If multiple Macs are required, Weave chooses how many transformer layers each machine should execute.

For example, a simple equal split of an 80-layer model would be:

- Mac A: 27 layers
- Mac B: 27 layers
- Mac C: 26 layers

But if Mac C is much slower, it becomes the bottleneck.

Weave might instead choose:

- Mac A: 42 layers
- Mac B: 25 layers
- Mac C: 13 layers

The goal is approximately to keep the pipeline balanced **while making sure every device has enough memory for the model layers and KV cache.**

Step 6 — Re-evaluate when conditions change

We do **not** initially plan to move model layers in the middle of generating every token.

That would introduce significant complexity involving KV state, synchronization, model-weight movement, and active requests.

Instead, Weave will make changes at safe points, such as:

- before a new request,
- between batches,
- or between conversation turns.

For example:

Earlier

2K context → A: 40 / B: 25 / C: 15

Later

32K context and less free memory on Mac C → A: 45 / B: 27 / C: 8

The scheduler can decide that the next execution should use the new configuration.

9. Main Research Questions

The project will focus on three questions.

RQ1 — Does live workload information improve model placement?

Can a scheduler that considers context length, concurrency, current memory, and measured stage performance outperform equal splitting and static hardware-based splitting?

This is the primary research question.

RQ2 — When is distributed inference actually useful?

For different models, contexts, network conditions, and workloads, when should an LLM use one Mac, two Macs, or three Macs?

We want to produce a practical map showing where distribution helps and where communication overhead makes it worse.

RQ3 — Can the scheduler adapt when conditions change?

If available memory, workload, or device performance changes, can Weave detect this and choose a better configuration for subsequent requests without manually restarting and redesigning the cluster?

This tests whether the system is genuinely adaptive rather than simply producing a good configuration once.

10. Expected Research Contribution

The project is successful if we can demonstrate that:

A model partition chosen using the actual workload and live runtime state performs better or avoids failures in situations where simpler static partitioning makes a poor decision.

For example, suppose four approaches are compared:

Baseline 1 — Equal split

Every Mac gets approximately the same number of layers.

Baseline 2 — Memory-proportional split

A Mac with twice the RAM receives roughly twice as many layers.

Baseline 3 — Static performance-aware split

A benchmark is performed when the cluster starts, and the layer allocation is determined from the result.

Weave — Runtime/workload-aware split

The scheduler additionally considers:

- context,
- concurrency,
- current available memory,
- actual stage timings,
- current workload,
- and previous execution measurements.

If Weave consistently produces better throughput/latency, avoids OOM situations, or makes better decisions about when to use fewer machines, we have demonstrated the value of the approach.

A negative result would also be useful: if adaptation helps only in certain workload ranges, we can identify those ranges and explain **when dynamic scheduling is worth its overhead**.

11. Experimental Evaluation

We will evaluate the system on two or preferably three different Apple Silicon Macs.

The experiments will deliberately vary conditions rather than simply reporting one tokens-per-second number.

Hardware configuration

Ideally the cluster should contain machines with different:

- Apple Silicon generations or tiers,
- memory capacities,
- memory bandwidth,
- and compute capability.

A heterogeneous cluster is preferable because heterogeneity is the problem being studied.

Workloads

We will test approximately:

- short context,
- medium context,
- long context,
- one simultaneous request,
- several simultaneous requests,
- idle machines,
- and machines experiencing controlled background load or memory pressure.

Models

We will select approximately two or three representative MLX-supported models.

At least one should be large enough that running it across multiple machines provides a genuine capacity advantage.

Metrics

The main measurements will include:

- **Time to First Token (TTFT)**
- **Time Per Output Token / inter-token latency**
- **tokens per second**
- **aggregate throughput under multiple requests**
- **P95 request latency**
- **memory usage per Mac**
- **number of out-of-memory failures**
- **scheduler decision time**

- **time required to change a partition**

We will also measure whether the scheduler's predicted best configuration matches the best configuration found experimentally.

12. Baselines and Existing Systems

We do not need to reproduce every distributed inference system available.

The most useful baselines are:

Single-node MLX-LM

Shows whether using multiple Macs is worthwhile at all.

Standard MLX distributed execution

Provides the Apple-native distributed baseline. MLX already supports distributed communication through RING and JACCL.

Exo

Provides a strong practical comparison because it already performs topology-aware distributed inference on Apple Silicon.

Internal placement baselines

The most important comparison for our research question will be:

Equal → RAM proportional → static performance aware → Weave adaptive.

This allows us to isolate whether **our scheduling information actually provides value**.

Systems such as dnet and prima.cpp will also be important related work even if reproducing all of them experimentally is not practical.

13. Technology Stack

The revised project will intentionally use a small technology stack.

Hardware

Apple Silicon Macs only.

Inference framework

MLX / MLX-LM

Distributed communication

Primary options:

- **JACCL / RDMA over Thunderbolt 5**, where supported
- **MLX RING over TCP/Thunderbolt/Ethernet** as fallback and comparison.

Apple's MLX documentation supports both approaches.

Parallelism

Primary research will use **pipeline parallelism**.

Tensor parallelism may be used only as an existing baseline or later extension. We will not attempt to invent and optimize every parallelism strategy.

Weave components

We expect to implement four main pieces:

1. **Cluster Profiler**
2. **Workload/Memory Estimator**
3. **Adaptive Placement Scheduler**
4. **Benchmarking and Runtime Monitoring Layer**

This is substantially smaller and more realistic than building an entirely new distributed inference framework.

14. What We Are Explicitly Not Trying to Solve

To keep the master's project feasible, the following are outside the primary scope:

- NVIDIA GPUs,
- Jetson boards,
- Windows/Linux clusters,
- iPhones and iPads,
- custom distributed networking,
- new model quantization algorithms,
- privacy-preserving cryptographic inference,
- new transformer architectures,
- distributed training,
- custom Metal kernels,
- arbitrary Internet-scale peer-to-peer inference,
- and seamless failure recovery from every possible hardware failure.

These may be interesting research problems, but they are separate projects.

The focus of Weave is:

Better runtime decisions for local distributed inference across a small heterogeneous cluster of Apple Silicon Macs.

15. Optional Stretch Goal: Failure and Stall Recovery

If the primary scheduler is completed ahead of schedule, we can investigate one additional problem.

Distributed inference systems can fail if one device crashes, disconnects, or becomes stuck.

Instead of trying to maintain full redundant copies of every model layer, we could investigate a simpler recovery mechanism:

1. detect a stalled node,
2. stop the affected pipeline,
3. recompute a valid partition using the remaining machines,
4. reconstruct required KV state from the stored conversation,
5. and resume the session.

This would be a useful secondary contribution, but **the project should still be considered successful without it.**

The scheduler remains the main research contribution.

16. Proposed Implementation Plan

Month 1 — Reproduce the Current State of the Art

Before building Weave, we will reproduce distributed inference using the actual hardware available to us.

Tasks:

- configure MLX,
- configure distributed MLX,
- test JACCL/RING where available,
- run Exo,
- run selected models,
- measure one-, two-, and three-Mac performance,
- understand existing placement behavior.

Milestone:

A reproducible benchmark showing how our actual Mac cluster behaves today.

This is important because the final scheduler design should be based on measurements rather than assumptions.

Month 2 — Profiling and Measurement System

Build the infrastructure for measuring:

- free memory,
- per-device layer/stage time,
- network performance,
- context length,
- concurrency,
- and KV-cache estimates.

Milestone:

For any request, Weave can describe the current state of the cluster and workload.

Month 3 — Static Placement Optimizer

Implement:

- equal split,
- memory-proportional split,
- measured-performance split,
- memory constraints.

Milestone:

Automatically produce valid unequal layer assignments for heterogeneous Macs.

Month 4 — Runtime/Workload-Aware Scheduler

Add:

- context-aware memory prediction,
- concurrency awareness,
- live stage timing,
- current memory headroom,
- decision between one/two/three Macs,
- re-evaluation at safe request boundaries.

Milestone:

The partition changes when workload or runtime conditions change.

Month 5 — Experiments and Adaptation

Run controlled experiments involving:

- longer contexts,
- increasing concurrency,
- artificial background workload,
- memory pressure,
- different network configurations.

Improve the scheduler based on the results.

If the main system is stable, begin the optional reliability component.

Month 6 — Full Evaluation

Freeze major features.

Run all:

- baseline experiments,
- scheduler ablations,
- model configurations,
- context configurations,
- concurrency experiments,
- and hardware configurations.

Produce final figures and tables.

Month 7 — Buffer, Thesis, and Demonstration

Focus on:

- final analysis,
- documentation,
- reproducibility,
- thesis/report writing,
- final presentation,
- and demonstration.

No large new features should be introduced during this period.

17. Team Division

For a team of three, responsibilities can be divided approximately as follows:

Member

Primary Responsibility

Member 1 MLX distributed execution, pipeline integration, model execution

Member 2 Profiling, performance model, placement optimizer, adaptive scheduler

Member 3 Benchmark framework, telemetry, workload generation, experiments, optional recovery system

All members will contribute to integration, experiments, analysis, and the final report.

18. Feasibility

The revised project is significantly more feasible than the original proposal.

The original plan required us to build networking, multiple inference backends, heterogeneous hardware support, quantization research, privacy mechanisms, and fault tolerance within the same project. The implementation plan assigned several large research areas only one to two months each.

In the revised design, Apple and MLX provide the difficult low-level distributed infrastructure.

Our work is concentrated on:

measurement → prediction → placement → adaptation → evaluation.

For a three-person team with approximately six to seven months, this is a realistic scope provided that we obtain consistent access to at least two, preferably three, Apple Silicon Macs.

Hardware availability should therefore be confirmed before final approval of the project.

19. Why This Is Still a Meaningful Master's Project

The value of the project is no longer that we can demonstrate an LLM running across several computers.

That has already been demonstrated.

The interesting question is now:

How should a small local AI cluster make intelligent runtime decisions when its machines and workloads are unequal and changing?

The project combines:

- distributed systems,
- machine-learning inference,
- scheduling,
- performance modeling,
- memory management,
- experimental systems research,
- and Apple Silicon/MLX.

Most importantly, the project produces a hypothesis that can be tested:

Using current workload and live runtime information will produce better placement decisions than using only static hardware information.

We can then prove, reject, or qualify that hypothesis through controlled experiments.

That gives the project a clear research contribution rather than simply an engineering demonstration.

20. Proposed Final Problem Statement

Problem Statement

Modern tools such as MLX, Exo, dnet, and related systems have made distributed LLM inference across consumer Apple Silicon machines increasingly practical. However, distributing a model does not automatically produce an efficient configuration, particularly when the machines have different compute capabilities and memory sizes and when workload characteristics change over time.

Existing Apple-focused systems already address important aspects of heterogeneous placement using device resources, model properties, network topology, and memory information. What remains less explored in practical multi-Mac inference is closed-loop placement that combines these relatively static properties with live workload and runtime information such as context length, concurrency, current memory headroom, and observed pipeline-stage performance.

Weave will investigate whether these signals can be used to dynamically choose both the number of participating Macs and the distribution of model layers between them. The system will build on MLX Distributed rather than implementing distributed execution from scratch and will re-evaluate placement at safe request boundaries as conditions change.

The project will compare this approach against equal partitioning, memory-proportional partitioning, static profiling, single-device MLX, and existing distributed approaches. The goal is to determine when runtime-adaptive placement improves performance or reliability, when static placement is sufficient, and when distributing an LLM across additional Macs is counterproductive.

21. Proposed Contribution in One Sentence

Weave investigates whether an Apple Silicon LLM cluster can make better model-placement decisions by adapting to the workload and the machines' real runtime conditions instead of relying only on a fixed hardware-based partition.

And in even simpler language:

Existing tools know how to make several Macs run one AI model together. We want to make the Macs decide intelligently how to share the work as the workload and the condition of the machines change.